femtoRV32

Single-Cycle RISC-V Processor

Design, implementation and verification report
2 November 2026

Kareem Abdellatif
Mohamed Aboelfotouh
Moustafa Fayed

A 32-bit educational single-cycle processor with separate instruction and data memories. Normal instructions pass through combinational fetch, decode, execution, memory access and writeback within one clock interval. Architectural state updates on the rising edge. ECALL and EBREAK stop PC advancement in this project execution model.

Verification snapshot	Fresh run of current sources
Directed coverage	40 RV32I + 8 RV32M instructions; FENCE.TSO and PAUSE additionally checked
Simulation	269 passing checks; zero failed checks; 110 cycles to ECALL
Execution bounds	Main program stops at byte PC 468; separate EBREAK reset test
Tools	Icarus Verilog 12.0; Yosys 0.52; Graphviz

Contents

1. Datapath architecture and block diagram
2. Implementation and control organization
3. Memory and system implementation
4. Verification strategy and results
5-6. Team waveform screenshot and test interpretation
7. Integration issues, assumptions and limitations
8. Reproduction commands
Appendix A. Original team block diagram
Appendix B. Original datapath schematic
Appendix C. Original GTKWave screenshot

Evidence comes from the current RTL and a fresh simulation. The waveform screenshot is the team-owned existing GTKWave image, reproduced directly from Testing/waveforms/waveform.png. Processor RTL and the main testbench were not modified for documentation.

1 / Datapath architecture

The instruction selects source/destination registers, generates an immediate and drives the main control and ALU decoder. The datapath uses two operand muxes, a four-source writeback mux and a dedicated comparator for signed/unsigned branch decisions.

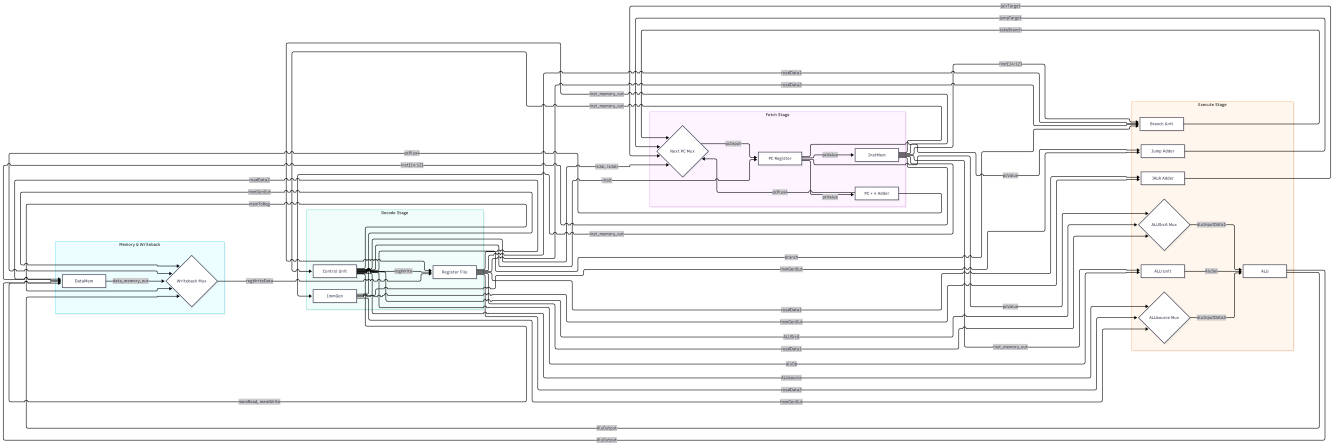


Figure 1. Original team block diagram, reproduced directly from Design/Single-Cycle_Block_Diagram.png. No blocks, wiring or labels have been redrawn. Appendix A reproduces the same image at a larger page size for zooming; Appendix B contains the existing Design/datapath_schematic.pdf.

PC routing and writeback

```
pcPlus4    = pcValue + 4
jumpTarget = pcValue + immGenOut
jalrTarget = (readData1 + immGenOut) & ~1
pcInput    = isJalr ? jalrTarget :
             (isJal || takeBranch) ? jumpTarget : pcPlus4
```

JAL/JALR write PC+4 to rd. LUI writes the upper immediate directly. AUIPC selects the current PC as ALU input A and the immediate as input B. The PC register is enabled by ~halt. Branch immediates are already byte-scaled by ImmGen; no additional shift is applied.

2 / Implementation

Module	Responsibility
datapath.v	Connects fetch, register file, operand muxes, ALU, memory, PC and writeback routing.
control_unit.v / defines.vh	Safe control defaults, opcode classes and shared ALU/writeback constants.
alu_unit.v / ALU.v	funct3/funct7 decoding; 5-bit ALU selection covers base operations and eight RV32M operations.
ImmGen.v / BranchUnit.v	I/S/B/U/J immediates; six signed/unsigned branch conditions.
RegFile.v	Combinational read ports and rising-edge writes. x0 is constant, without writable storage.
InstMem.v / DataMem.v	128 instruction words and 64 data words, initialized with the directed program and test data.
N_bit_register.v / DFlipFlop.v / mux2x1.v	Structural load-enabled registers with asynchronous reset.

Control encoding

Signal	Meaning
aluOp[1:0]	00: address/add; 01: branch subtraction; 10: register/RV32M; 11: immediate ALU.
memToReg[1:0]	00: ALU; 01: memory; 10: PC+4; 11: immediate.
aluSrcA / aluSource	Select PC vs rs1, and immediate vs rs2, respectively.
branch / isJal / isJalr	Select branch comparison and jump routing.

RV32M arithmetic

High-product multiplication distinguishes signed-signed, signed-unsigned and unsigned-unsigned operands. Division/remainder explicitly handle zero divisors and INT_MIN / -1 overflow. DIV/DIVU by zero return FFFFFFFF; REM/REMU by zero return the dividend. Signed overflow returns INT_MIN for DIV and zero for REM. These operators are combinational; timing closure has not been measured.

3 / Memory and system implementation

The processor supports byte, halfword and word memory accesses, together with system and synchronization instructions. This section describes memory lane selection, extension/masking and system control in the integrated team design.

Byte and halfword loads

DataMem selects mem[addr[7:2]]. addr[1:0] selects a byte; addr[1] selects the lower or upper halfword. LB/LH repeat the selected sign bit. LBU/LHU zero-fill the upper bits. The organization is little-endian.

Operation	Selected value / address	Expected load
LB	B2 at byte 20	FFFFFFB2
LB	7C at byte 23	0000007C
LBU	B2 at byte 20	000000B2
LH	F1B2 at byte 20	FFFFF1B2
LH	7C8A at byte 22	00007C8A
LHU	F1B2 at byte 20	0000F1B2

Partial-word stores

SB replaces one byte with dataIn[7:0]. SH replaces the selected halfword with dataIn[15:0]. Unaddressed bytes retain their old values. Four SB operations build 0x0AFB140A in mem[6]; two SH operations build 0x0014FFFB in mem[7]. LW reads both words back. The source word mem[5] remains 0x7C8AF1B2.

System and synchronization

FENCE/FENCE.TSO/PAUSE leave side-effect controls disabled and advance normally. SYSTEM decoding recognizes ECALL/EBREAK from funct3, rd, rs1 and upper immediate fields and asserts halt. Neither instruction writes registers or memory. A CSRRW encoding is checked to ensure it does not incorrectly halt; CSR execution itself is unsupported.

4 / Verification and results

The testbench uses a 20 ns clock period. A rising-edge monitor records the instruction index and independent expected values, waits 1 ns for state updates, then checks the next PC and register/memory result. Negative-edge logging produces a cycle trace. Transient results are saved before later instructions reuse the registers.

Test family	Observed result
Main program	110 cycles until ECALL; PC remains 468 for five additional rising edges.
Branch / control flow	Score 70; trap count 0. All six branch conditions, an unsigned not-taken case, jump links and skipped traps pass.
Loads / stores	Sign/zero extension, all byte lanes, both halfword positions and LW readback pass.
Base ALU / upper immediate	Register ALU, immediate ALU/shifts, LUI, AUIPC, JAL and JALR checks pass.
RV32M	Eight operations tested for (-1,-1), (-20,7), (-20,0) and (INT_MIN,-1).
Synchronization / SYSTEM	FENCE/FENCE.TSO/PAUSE decode and execution checks pass; ECALL and separate EBREAK halt tests pass.
Coverage words	RV32I low word FFFFFFFF; RV32I upper byte FF; RV32M byte FF.

Fresh run summary

```
Passing [ OK ] checks: 269
Failed [FAIL] checks:  0
Cycles until halt:     110
ECALL byte PC:         468
SIMULATION PASSED:
All 40 RV32I + 8 RV32M instructions checked!
```

Evidence: Testing/documentation_test_results.txt and Testing/datapath.vcd. Yosys hierarchy elaboration and RTL schematic generation also complete successfully; the log is Testing/documentation_yosys.log.

The base count includes FENCE/ECALL/EBREAK. FENCE.TSO and PAUSE share the FENCE coverage class and also have separate decode checks. Set coverage bits establish directed instruction coverage, not exhaustive ISA compliance, formal proof or physical timing closure.

5 / Team waveform screenshot

The following is the existing GTKWave screenshot saved by the team in Testing/waveforms/waveform.png. It is reproduced without redrawing traces or changing signal values. Appendix C provides the same screenshot on a larger page for zooming.

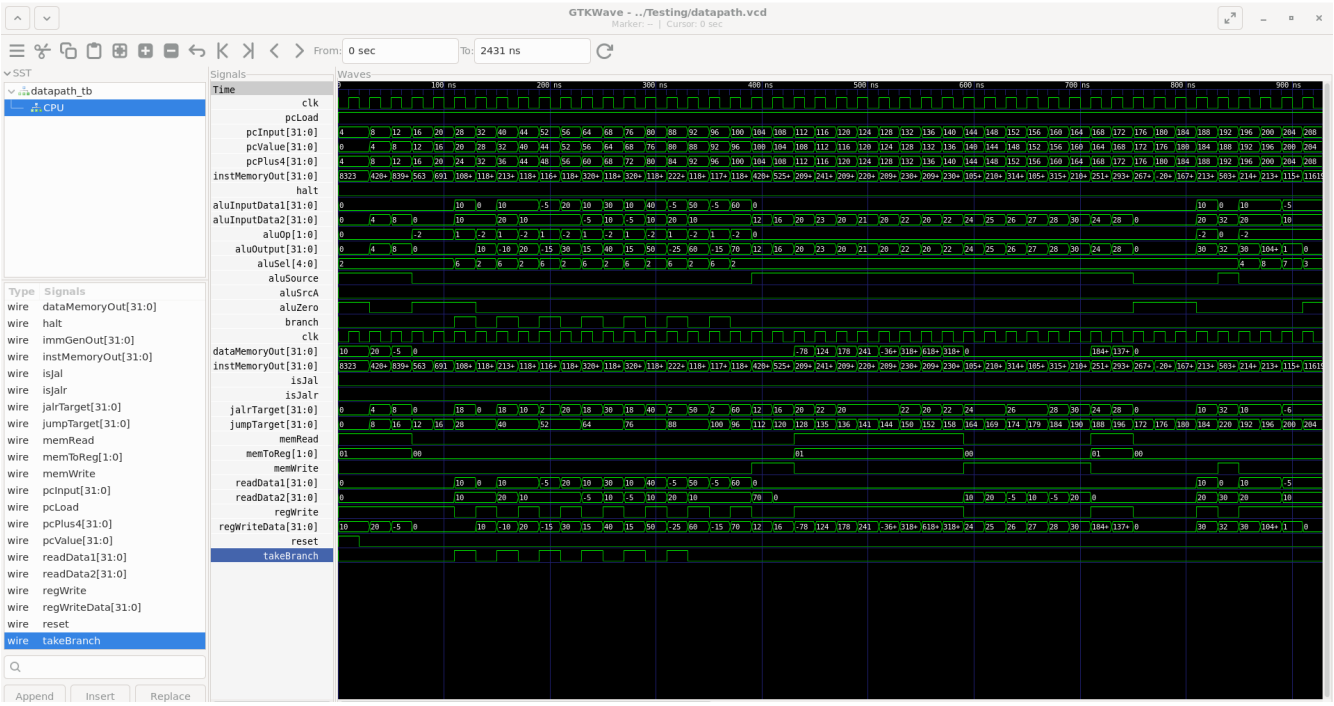


Figure 2. Original GTKWave screenshot of Testing/datapath.vcd. The visible window covers approximately 0-930 ns. The signal panel includes the PC, instruction, ALU inputs/output, load/store controls, writeback and branch decision. GTKWave displays several buses in decimal; narrow bus values are abbreviated in the original screenshot.

Visible load/store behavior

In the visible load sequence, dataMemoryOut and regWriteData include -78 (LB of 0xB2), 124 (LB of 0x7C), 178 and 241 (LBU). LH/LHU results follow. memRead asserts for loads. The subsequent store sequence asserts memWrite while aluOutput addresses the selected byte and halfword lanes. These observations are backed by the exact result checks in the simulation log.

Visible control flow

The early PC sequence skips trap instructions after taken branches. takeBranch pulses align with branch decisions. The screenshot also shows normal PC progression through the synchronization instructions, followed by the subsequent arithmetic/store operations.

6 / Interpreting the project evidence

The screenshot is an overview of the early program. Its visible range does not show the final ECALL or the separate EBREAK phase. Those outcomes are established by the saved self-checking simulation results and by the complete VCD, rather than being inferred from an unseen part of the image.

Evidence	What it establishes
Original GTKWave screenshot	Early PC/branch behavior, memory-read/write controls, ALU activity and register writeback in the displayed time range.
documentation_test_results.txt	Exact load/store expected values, branch score 70, trap count 0, all instruction coverage checks and final PASS.
datapath.vcd	Complete captured signal trace, including later RV32M execution, ECALL halt and the separate EBREAK test.
Original block diagram	The team-owned proposed datapath structure, routing and signal labels.
Original datapath schematic	Existing Yosys module-level connectivity, preserved as a vector PDF.

ECALL and EBREAK checks

The main program reaches byte PC 468 with instruction 0x00000073. halt asserts and pcLoad clears. Five additional rising edges leave the PC unchanged. The testbench then disables its main monitor, asserts reset and places 0x00100073 at instruction word zero. After reset release, five rising edges leave the PC at zero with halt asserted and no register write.

Memory results checked in the log

```
LB  byte 0:        FFFFFFB2
LBU byte 0:        000000B2
LH  lower half:    FFFFF1B2
LHU lower half:    0000F1B2
SB word readback:  0AFB140A
SH word readback:  0014FFFB
```

The figures in this revision are taken from the team-owned saved artifacts. No new datapath layout, substitute schematic or reconstructed waveform is included.

7 / Issues and solutions

Issue	Resolution in the integrated implementation
Duplicate net / control widths	Keep one two-bit cu_memToReg and an explicit two-bit cu_aluOp; compare the full ten-bit side-effect control vector.
Unknown module types	Compile all RTL files together from Verilog/ so dependencies and defines.vh resolve.
Instruction-test gaps	Add SUB, AND, OR and JAL cases; check individual results and next PC as well as final state.
Reused destination registers	Save transient LB/SLT/SLTU/SLTI values at the relevant instruction boundary.
Writes to register zero	Generate writable registers only for x1-x31; assign d_reg[0] to constant zero.
RV32M corner cases	Explicit divide-by-zero and signed-overflow handling supplies deterministic results.

Assumptions and known limitations

Memory is little-endian: 512 instruction bytes and 256 data bytes. High address bits are discarded and therefore alias. Aligned halfword/word accesses and 32-bit instruction alignment are assumed; cross-word accesses and misalignment traps are absent.

The processor has no CSRs, interrupts, privilege modes, virtual memory or illegal-instruction trap. Decoders use inst[6:2] and do not fully validate every encoding. Unsupported SYSTEM operations are effectively ignored. Compressed, atomic and floating-point instructions are absent.

FENCE/FENCE.TSO/PAUSE are NOPs in the single-hart, unbuffered memory model. ECALL/EBREAK stop the PC rather than invoking a handler. Data memory initializes once and is not cleared by CPU reset.

The board wrapper is commented out. FPGA resource use, clock frequency, timing closure and hardware operation are not demonstrated. Combinational RV32M division may impose a long critical path. The top exposes only clock/reset and is observed through internal simulation signals.

8 / Reproduce the evidence

Run from the Verilog directory in WSL. Icarus compiles/simulates the design. Yosys elaborates the RTL and Graphviz renders the schematic. The additional capture module only observes signals.

```
cd "/mnt/d/New folder/architecure-project/Verilog"

iverilog -g2012 -Wall -s datapath_tb -s waveform_capture \
  -o ../Testing/waveform_sim.out *.v \
  ../Testing/waveform_capture.v
vvp ../Testing/waveform_sim.out \
  | tee ../Testing/documentation_test_results.txt
```

The following Bash source-list variable keeps the Yosys command readable without changing the module hierarchy:

```
rtl="datapath.v DataMem.v InstMem.v RegFile.v"
rtl="$rtl Nbit_decoder.v N_bit_register.v DFlipFlop.v"
rtl="$rtl mux2x1.v ImmGen.v control_unit.v alu_unit.v"
rtl="$rtl ALU.v BranchUnit.v"

yosys -p "read_verilog $rtl; hierarchy -check -top datapath;
proc; show -format dot \
-prefix ../Design/report_datapath_schematic datapath"

dot -Tpdf ../Design/report_datapath_schematic.dot \
  -o ../Design/report_datapath_schematic.pdf
# Optional viewer, if installed:
gtkwave ../Testing/datapath.vcd
```

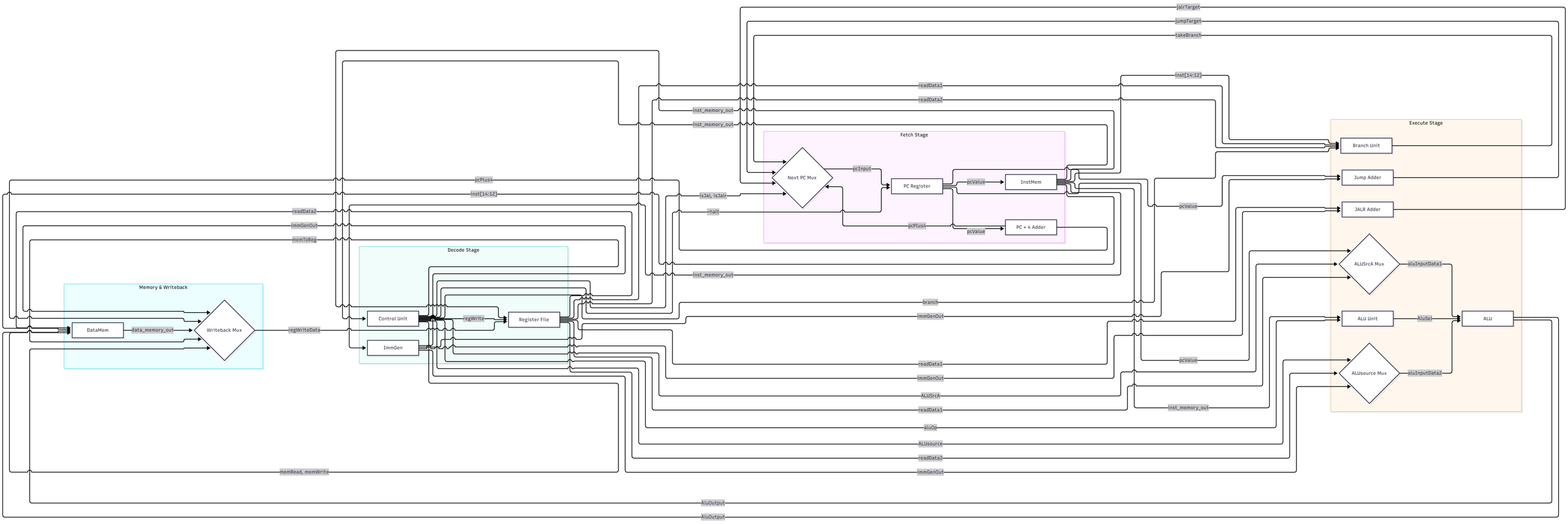
Regenerate the document figures

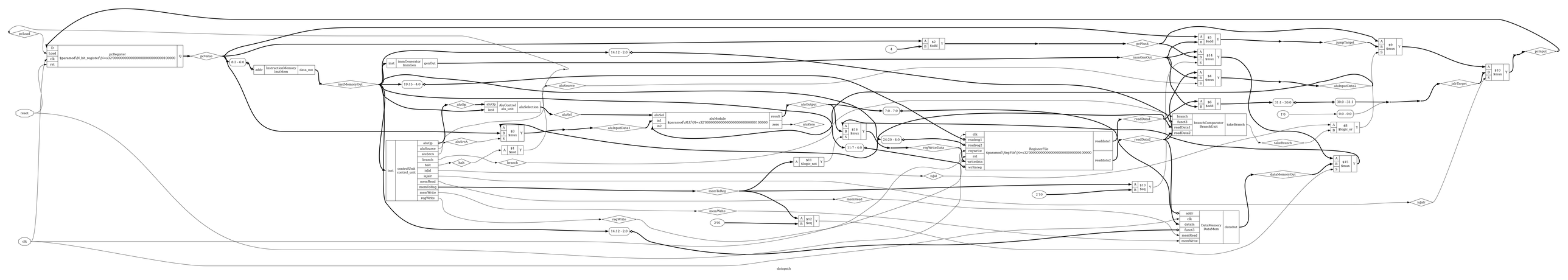
Design/build_report.py assembles the existing team diagrams, waveform screenshot and verified project text using ReportLab, Pillow and pypdf. It does not redraw the datapath or waveform. Design/build_readmes.py regenerates the project README files.

Appendices / Original project diagrams

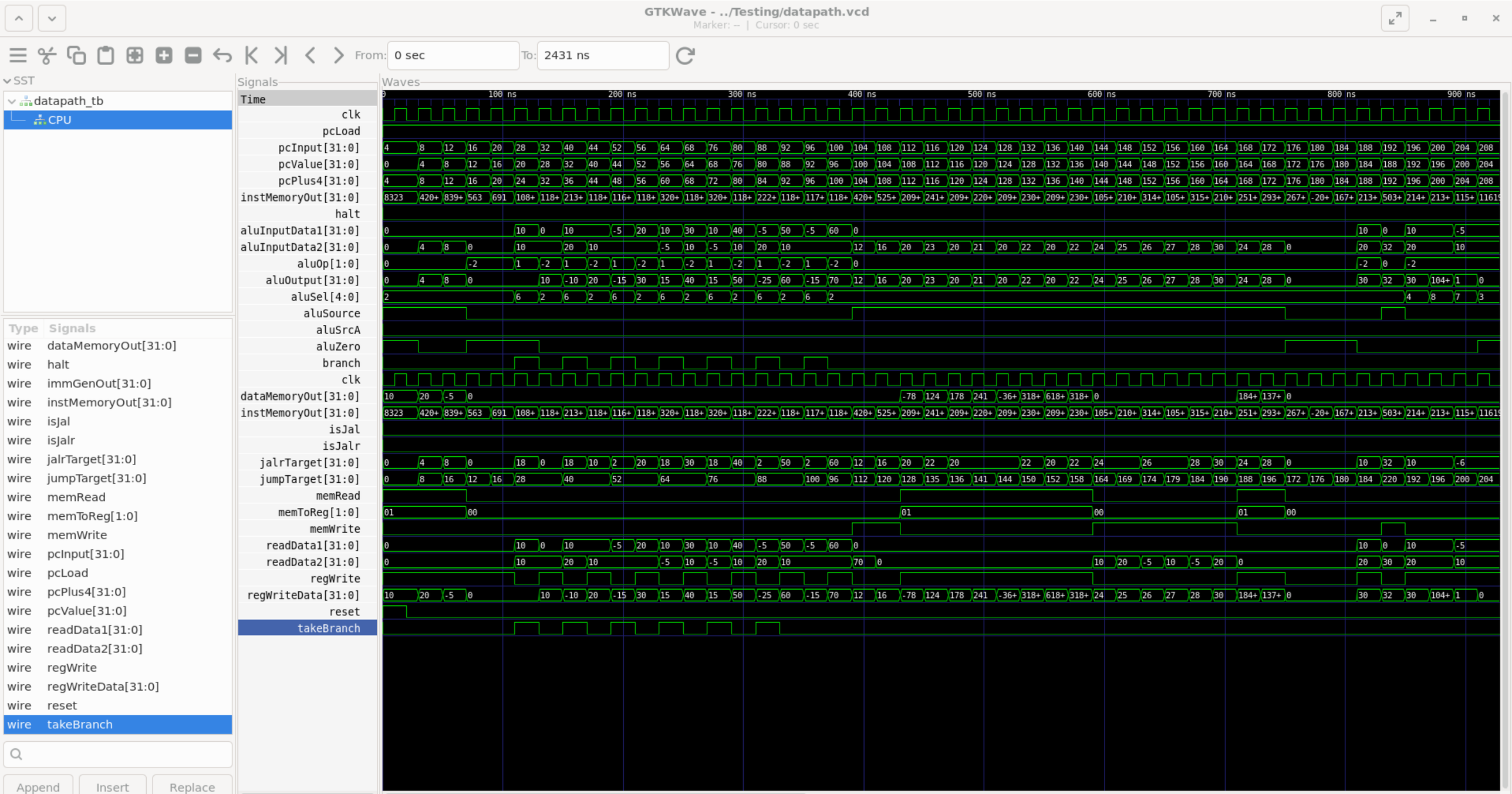
Appendix A reproduces the existing Single-Cycle_Block_Diagram.png without redrawing it. Appendix B directly includes the existing datapath_schematic.pdf as a vector page. Both are taken from the team Design folder. Appendix C reproduces the original GTKWave screenshot from Testing/waveforms/waveform.png. Use PDF zoom to read the wide drawings. The schematic describes RTL hierarchy, not a routed gate-level implementation.

Included source artifacts: Design/Single-Cycle_Block_Diagram.png (also described in Design/diagram.md) and Design/datapath_schematic.pdf (with Design/datapath_schematic.svg as its companion). The original images and source files are unchanged. No substitute diagram is used.


jalrTarget
jumpTarget
takeBranch
inst[14:12]
readData1
readData2
inst_memory_out
inst_memory_out
Fetch Stage
Next PC Mux
pcInput
PC Register
pcValue
InstMem
pcPlus4
PC + 4 Adder
pcValue
Execute Stage
Branch Unit
Jump Adder
JALR Adder
pcPlus4
inst[14:12]
isJal, isJalr
halt
pcValue
readData2
immGenOut
memToReg
Decode Stage
ALUSrcA Mux
aluInputData1
inst_memory_out
Memory & Writeback
Control Unit
regWrite
Register File
branch
immGenOut
DataMem
data_memory_out
Writeback Mux
regWriteData
ImmGen
ALU Unit
AluSel
ALU
pcValue
readData1
immGenOut
ALUSrcA
ALUsource Mux
aluInputData2
inst_memory_out
readData1
aluOp
ALUsource
readData2
immGenOut
memRead, memWrite
AluOutput
AluOutput


pcLoad
reset
clk
pcRegister
$paramod\N_bit_register\N=s32'00000000000000000000000000100000
D
Load
clk
rst
Q
pcValue
8:2 - 6:0
addr
InstructionMemory
InstMem
data_out
instMemoryOut
inst
ImmGenerator
ImmGen
genOut
14:12 - 2:0
19:15 - 4:0
4
$2
$add
Y
pcPlus4
immGenOut
$5
$add
$14
$mux
$4
$mux
$6
$add
jumpTarget
$9
$mux
jalrTarget
$10
$mux
pcInput
aluSource
aluInputData2
31:1 - 30:0
30:0 - 31:1
1'0
0:0 - 0:0
$8
$logic_or
takeBranch
aluOp
aluOp
inst
AluControl
alu_unit
aluSelection
aluSel
aluSel
in1
in2
aluModule
$paramod\ALU\N=s32'00000000000000000000000000100000
result
zero
aluZero
aluOutput
$16
$mux
7:0 - 7:0
24:20 - 4:0
11:7 - 4:0
$3
$mux
aluSrcA
$1
$not
aluInputData1
branch
halt
isJal
isJalr
memRead
memToReg
memWrite
regWrite
controlUnit
control_unit
$11
$logic_not
regWriteData
RegisterFile
$paramod\RegFile\N=s32'00000000000000000000000000100000
clk
readreg1
readreg2
regwrite
rst
writedata
writereg
readdata1
readdata2
readData1
readData2
branch
funct3
readData1
readData2
branchComparator
BranchUnit
takeBranch
$12
$eq
2'01
$13
$eq
2'10
$15
$mux
dataMemoryOut
addr
clk
datain
funct3
memRead
memWrite
DataMemory
DataMem
dataOut

datapath


GTKWave - ../Testing/datapath.vcd
Marker: -- | Cursor: 0 sec
From: 0 sec
To: 2431 ns
SST
datapath_tb
CPU
Type Signals
wire dataMemoryOut[31:0]
wire halt
wire immGenOut[31:0]
wire instMemoryOut[31:0]
wire isJal
wire isJalr
wire jalrTarget[31:0]
wire jumpTarget[31:0]
wire memRead
wire memToReg[1:0]
wire memWrite
wire pcInput[31:0]
wire pcLoad
wire pcPlus4[31:0]
wire pcValue[31:0]
wire readData1[31:0]
wire readData2[31:0]
wire regWrite
wire regWriteData[31:0]
wire reset
wire takeBranch
Append
Insert
Replace
Signals
Time
clk
pcLoad
pcInput[31:0]
pcValue[31:0]
pcPlus4[31:0]
instMemoryOut[31:0]
halt
aluInputData1[31:0]
aluInputData2[31:0]
aluOp[1:0]
aluOutput[31:0]
aluSel[4:0]
aluSource
aluSrcA
aluZero
branch
clk
dataMemoryOut[31:0]
instMemoryOut[31:0]
isJal
isJalr
jalrTarget[31:0]
jumpTarget[31:0]
memRead
memToReg[1:0]
memWrite
readData1[31:0]
readData2[31:0]
regWrite
regWriteData[31:0]
reset
takeBranch
Waves
0
100 ns
200 ns
300 ns
400 ns
500 ns
600 ns
700 ns
800 ns
900 ns